

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Final Group Project Report submitted for BDA501 – Big Data

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Final Group Project Report submitted for BDA501 – Big Data

Supervisor:

TAN Le Duy

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Degree Master of Software Engineering

FPT University

2026

Abstract

This thesis presents an academic fraud risk scoring workflow for the IEEE-CIS dataset. Apache Spark performs typed ingestion, transaction–identity joining, quality auditing, chronological leakage-controlled processing, feature engineering, and Parquet handover. Python model training compares Logistic Regression, LightGBM, XGBoost, and CatBoost after the Spark-to-pandas boundary. The system packages feature mappings, calibration, thresholds, and explanations for an in-process FastAPI service with a React review interface.

The current CatBoost balanced candidate reports validation ROC-AUC 0.9081210 and PR-AUC 0.5805993, with holdout ROC-AUC 0.8780755 and PR-AUC 0.4266435. The current CatBoost candidate is labelled `v2` in its metadata, but its promotion decision is `not_promoted`; the configured V2 software default remains unchanged. The principal limitations are offline–online historical-feature mismatch, duplicated policy ownership, local lineage, single-node training, fail-open fallback, and incomplete security and monitoring controls. The result is a bounded academic demonstration, not a production-readiness claim.

ACKNOWLEDGMENTS

The members of RocketUpTeam sincerely thank supervisor TAN Le Duy for guidance and feedback provided throughout project development and documentation.

Team Roster and Repository Role Mapping

Table 1 lists the project team members, official roles, and primary repository responsibilities.

Table 1: *Team Roster and Repository Role Mapping*

Member Name	Role	Primary Responsibility
TUYEN Le Quang	Project Leader	Responsible for system ideation and overall project management.
NHI Nguyen Le Hong	Content & Presentation Coordinator	Responsible for synthesizing project content, designing slides, and developing the presentation strategy.
AN Duong Binh	Big Data & Pipeline Engineer	Responsible for Spark data ingestion, left joins, temporal splitting, and feature contract verification.
LONG Pham Duc	Machine Learning Engineer	Responsible for classifier training, hyperparameter tuning, isotonic calibration, and metric evaluation.
TRUNG Do Quoc	Backend & API Engineer	Responsible for FastAPI scoring service, database persistence, and scoring contracts.
QUAN Duong Hong	Frontend & UI Engineer	Responsible for React dashboard UI, risk visualizations, and analyst review workflow.

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Full Definition / Meaning
API	Application Programming Interface
AUC	Area Under the Curve
BDA	Big Data Analytics
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
FPT	Corporation / FPT University
GBDT	Gradient Boosted Decision Trees
HTTP	Hypertext Transfer Protocol
IEEE-CIS	IEEE Computational Intelligence Society
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
LR	Logistic Regression
ML	Machine Learning
MLlib	Machine Learning Library (Apache Spark)
PR-AUC	Precision-Recall Area Under the Curve
ROC-AUC	Receiver Operating Characteristic Area Under the Curve
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
UI	User Interface
V2	Version 2 Model Release Authority

CONTENTS

Abstract	3
Acknowledgments	4
List of Abbreviations and Acronyms	5
1 Introduction	11
1.1 Problem context	11
1.2 Objectives & research questions	11
1.3 Evaluation basis	11
1.4 BDA501 requirement compliance	11
2 Background and Related Work	13
2.1 Imbalanced temporal ranking	13
2.2 IEEE-CIS dataset structure	13
2.3 Technology boundary	13
3 Requirements and System Architecture	14
3.1 Functional requirements	14
3.2 Quality requirements	14
3.3 Integrated architecture	14
3.4 System contracts	15
3.5 Architectural tradeoffs	16
3.6 Main source code components	16
3.7 Distributed Spark execution evidence	16
4 Big-Data Processing and Feature Engineering	18
4.1 Source validation and typed ingestion	18
4.2 Join integrity	18
4.3 Leakage-controlled transformation order	18
4.4 Chronological split	19
4.5 Feature engineering & class balance	19
4.6 Verification result	20

5	Model Development, Evaluation, and Serving	21
5.1	Artifact identity and terminology	21
5.2	Model-development protocol	21
5.3	PySpark MLlib DecisionTree baseline	22
5.4	Candidate models and selection	22
5.5	Current CatBoost candidate results	22
5.6	Packaging, checksum, and promotion	22
5.7	Request-time scoring	23
6	Application and Deployment	24
6.1	Integrated serving architecture	24
6.2	Scoring and review workflow	24
6.3	Frontend interface	24
6.4	Deployment boundary	25
7	Validation and Whole-System Review	26
7.1	Evidence status	26
7.2	Technical challenges and implemented solutions	27
7.3	Consolidated limitations	28
8	Conclusion and Future Work	29
8.1	Conclusion	29
8.2	Summary of research question answers	29
8.3	Future work	29
	References	30
A	API Reference	31
A.1	Common enums	31
A.2	Core response fields	31
A.3	Endpoint summary	31
B	Deployment and Implementation Diagrams	33
B.1	Application deployment	33
C	Feature and Risk Contracts	34
C.1	Canonical feature groups	34

C.2	Forbidden model columns	34
C.3	Risk bands	35
D	Reproducible Runbook	36
D.1	Raw data placement	36
D.2	Preprocessing	36
D.3	Model training	36
D.4	Application	36
D.5	Local development	37
D.6	Rollback	37
E	Test and Contribution Matrix	38
E.1	Recommended validation commands	38
E.2	Acceptance scenarios	38
E.3	Team roster and repository role mapping	39

LIST OF FIGURES

3.1	Overall system architecture	15
3.2	Data, feature, and evaluation contracts	15
4.1	Detailed data processing pipeline	19
5.1	Model training lifecycle	21
5.2	Promotion state machine	23
6.1	Application User Interface: Transaction queue and real-time risk scoring break-down.	24
6.2	Application User Interface: Transaction inspection and analyst review interface.	25
7.1	Exploratory Data Analysis: Class distribution and fraud rate by ProductCD.	26
7.2	Temporal fraud association and model-family performance selection.	26
7.3	Candidate holdout evaluation: Metrics drop and operating confusion matrix.	27
7.4	Probability calibration: Raw vs isotonic-calibrated Brier score on holdout.	27
B.1	Default application deployment	33

LIST OF TABLES

1	Team Roster and Repository Role Mapping	4
1.1	BDA501 Requirement-Compliance Matrix	12
3.1	Functional requirements specification	14
3.2	Quality requirements specification	14
3.3	Major architectural tradeoffs	16
3.4	Main Source Files and Component Responsibilities	16
3.5	Distributed Spark Execution Evidence	17
4.1	Observed source files	18
4.2	Transaction–identity join audit	18
4.3	Chronological labeled splits	19
4.4	Training and evaluation class distributions	19
5.1	Model identifiers and runtime roles	21
5.2	PySpark MLlib DecisionTree Baseline Performance	22
5.3	Current model-family selection evidence	22
5.4	Current CatBoost candidate validation and holdout metrics	22
5.5	Current candidate promotion gate	23
7.1	Technical Challenges, Implemented Solutions, and Evidence	28
A.1	Core API fields	31
C.1	Canonical feature groups	34
E.1	Acceptance scenarios	38
E.2	RocketUpTeam Roster and Repository Role Mapping	39

CHAPTER 1

INTRODUCTION

1.1 Problem context

Online transaction fraud is an imbalanced classification problem with rare positive instances and asymmetric error costs [1, 2]. A production fraud scoring system requires reproducible data pipelines, temporal validation integrity, explainable predictions, and an operational human-review workflow.

How can a reproducible big-data pipeline, an imbalanced-classification model, an explainable scoring API, and a human-review interface be integrated into one auditable fraud risk scoring system?

1.2 Objectives & research questions

Core Objectives:

- Ingest and validate the **1.29 GB** IEEE-CIS transaction and identity dataset via Apache Spark.
- Construct leakage-free chronological dataset splits (`train`, `val`, `holdout`).
- Evaluate MLib DecisionTree, LightGBM, XGBoost, and CatBoost classifiers.
- Serve packaged models behind a FastAPI scoring API with TreeSHAP explanations and React UI.

Research Questions:

- RQ1:** Can raw data be processed without identifier overlap or temporal leakage?
- RQ2:** Which classifier achieves highest validation PR-AUC and retains holdout performance?
- RQ3:** Can the model be served efficiently without a Spark dependency at request time?
- RQ4:** What technical boundaries separate the academic demo from production readiness?

1.3 Evaluation basis

Primary metrics are ROC-AUC and Precision-Recall AUC (PR-AUC). PR-AUC is authoritative for rare positive classes [3, 4].

1.4 BDA501 requirement compliance

Table 1.1 summarizes compliance with BDA501 project rubric requirements.

Table 1.1: BDA501 Requirement-Compliance Matrix

Requirement	Project evidence
Main tool: Apache Spark	PySpark DataFrame & MLlib pipeline in Spark Preprocessing Module
Dataset size $\geq$ 500 MB	4 raw IEEE-CIS CSV files totaling 1.29 GB in raw dataset store
Raw-data ingestion	Explicit Spark schemas & required file validation in ingestion pipeline
Cleaning & transform	Stateless cleanup, identity normalization, missingness handling
Descriptive analysis	Spark <code>groupBy</code> , <code>agg</code> , <code>Window</code> analytics & EDA reporting module
Machine learning	PySpark MLlib DecisionTree baseline & CatBoost/LightGBM/XGBoost comparison
Distributed cluster	2 -worker Docker Compose Spark standalone cluster configuration
Architecture diagram	End-to-end system architecture & contract diagrams in documentation
Installation instructions	Project <code>README</code> and Docker Compose full-stack orchestration
Main source code	Modular source tree (<code>pipeline</code> , <code>model</code> , <code>backend</code> , <code>frontend</code>)
Analysis results	Saved JSON/Parquet metrics, PR-AUC, confusion matrix, calibration reports
Challenges & solutions	Summarized in Section 1.4 & Technical Challenges Matrix
Demo readiness	Full-stack Docker Compose deployment with React UI & FastAPI scoring backend

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 Imbalanced temporal ranking

Fraud detection is framed as binary temporal ranking on rare positive cases (fraud rate $\approx$ **3.5%**). Chronological splitting along `TransactionDT` preserves temporal order and prevents look-ahead leakage.

2.2 IEEE-CIS dataset structure

The dataset contains **590,540** training transactions and optional identity records [1]. Preserving transaction row authority via LEFT JOIN prevents data loss while capturing missingness indicators.

2.3 Technology boundary

Architecture operates via **distributed Spark data preparation with single-node Python estimator training**. CatBoost, LightGBM, XGBoost, and Logistic Regression are evaluated, using TreeSHAP for feature attribution [5].

CHAPTER 3

REQUIREMENTS AND SYSTEM ARCHITECTURE

3.1 Functional requirements

Table 3.1: *Functional requirements specification*

ID	Requirement Description
FR-01	Validate and process required IEEE-CIS transaction and identity CSV files.
FR-02	Produce train, validation, holdout, and unlabeled model-ready datasets.
FR-03	Preserve canonical feature order and preprocessing metadata.
FR-04	Compare binary classifiers using validation PR-AUC metrics.
FR-05	Score transactions and return probability, risk score, version, and explanations.
FR-06	Persist scored transactions and human review outcomes.
FR-07	Filter, sort, paginate, and inspect transaction records.
FR-08	Import CSV and load processed datasets with progress reporting.
FR-09	Support review approval/rejection and fraud/legitimate labels.
FR-10	Present risk, SHAP evidence, and model state in a browser interface.

3.2 Quality requirements

Table 3.2: *Quality requirements specification*

ID	Requirement Description
QR-01	Temporal leakage control and non-overlapping chronological splits.
QR-02	Reproducible schema and feature contracts.
QR-03	Version-aware model artifacts and rollback capability.
QR-04	Explainable tree-model predictions via TreeSHAP.
QR-05	Local containerized deployment with bounded data mounts.
QR-06	Automated subsystem test suite.
QR-07	Auditable experiment and promotion lineage.
QR-08	Durable background job operations.
QR-09	Role-based authentication, authorization, and audit logging.
QR-10	Real-time production latency and drift monitoring.

3.3 Integrated architecture

The system integrates batch data preparation, offline ML training, and online inference into an in-process modular monolith. Figure 3.1 presents the overall architecture and data flow contracts.

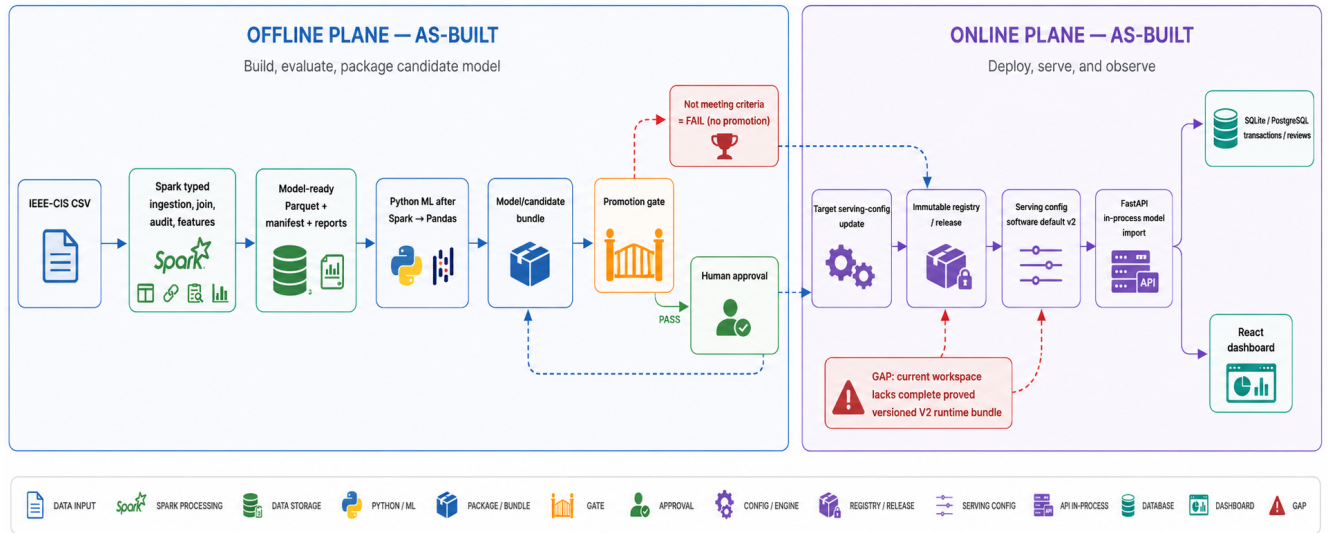


Figure 3.1: Overall system architecture. Solid lines are implemented; dashed lines are proposed or partially enforced. The failed-candidate path leaves serving unchanged.

3.4 System contracts

The system enforces three primary contracts:

1. **Data Contract:** Canonical 68-feature vector and schema hash.
2. **Scoring Contract:** Per-transaction request/response, probability, and TreeSHAP.
3. **API Contract:** Pydantic & TypeScript REST endpoints.

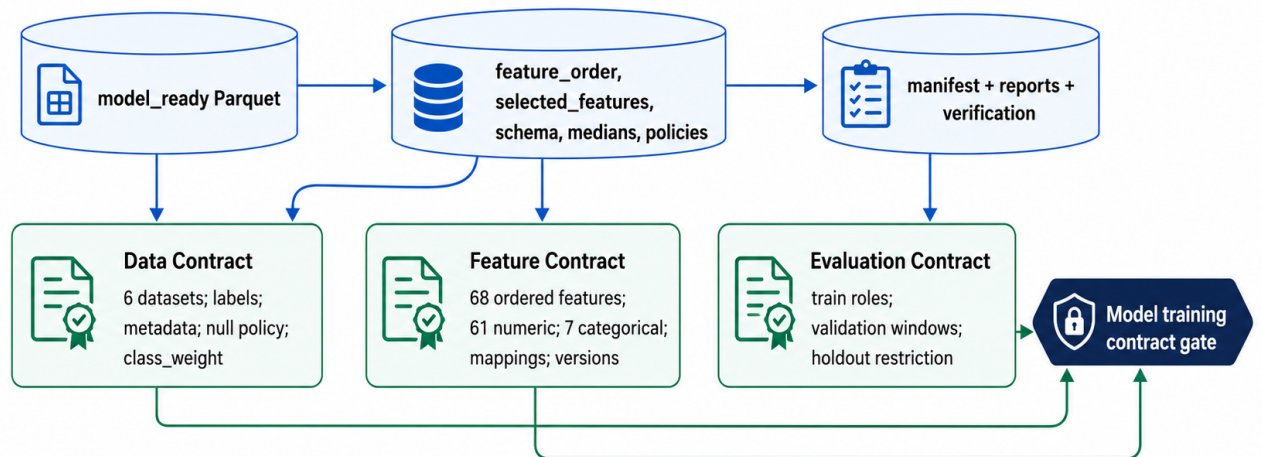


Figure 3.2: Data, feature, and evaluation contracts. Split configuration and training protocol define the Evaluation Contract.

3.5 Architectural tradeoffs

Table 3.3: *Major architectural tradeoffs*

Choice	Benefit	Cost
Monorepo and path dependency	Simple handoff and one stack	Backend inherits model dependencies
Spark then pandas	Scalable preparation and familiar estimators	Driver-memory training ceiling
Fixed backend risk bands	Stable UI and simple tests	Not tied to model operating evidence
In-memory jobs	Minimal infrastructure	Lost on restart and single-worker only
SQLite local default	Fast developer startup	Different operational behavior from PostgreSQL
Heuristic fallback	Demo continuity	Can conceal model loading failures
Manual TypeScript types	No generator setup	Contract drift risk

3.6 Main source code components

Table 3.4 outlines the primary source implementation files and core responsibilities across project components.

Table 3.4: *Main Source Files and Component Responsibilities*

Component	Module reference	Responsibility
Spark Preprocessing	Preprocessing Pipeline	Schema ingestion, left join, leakage-free transformation, Parquet export
Data Verifier	Verification Engine	Validates dataset row counts, schema hash, split overlap, verification report
Model Training	Estimator Training Module	Single-node candidate training, tuning, calibration, holdout evaluation
Backend Scoring	FastAPI Scoring Service	Artifact loading, probability scoring, SHAP calculation, FastAPI endpoints
Frontend Dashboard	React UI Dashboard	React UI, risk queue, SHAP feature rendering, analyst review workflow
Docker Deployment	Multi-container Orchestration	Multi-container service orchestration (backend, frontend, db, preprocessing)

3.7 Distributed Spark execution evidence

Table 3.5 presents the Spark execution configurations and deployment specification available in the repository.

Table 3.5: Distributed Spark Execution Evidence

Spark mode	Master URL	Wk	Cpu	Mem	Out	Deployment Spec
Local Preproc	local[4]	1	4	8GB	280MB	Preprocessing Service Spec
Cluster Mode	spark://master:7077	2	4	8GB	280MB	Standalone Cluster Spec

Note: Cluster configuration is implemented, but a clean end-to-end distributed runtime was not independently verified.

CHAPTER 4

BIG-DATA PROCESSING AND FEATURE ENGINEERING

4.1 Source validation and typed ingestion

The raw IEEE-CIS dataset contains **4** CSV files totaling **1.29 GB** (Table 4.1). The Spark pipeline applies explicit schemas and file discovery checks.

Table 4.1: *Observed source files*

File	Bytes	Role
train_transaction.csv	683,351,067	Labeled transactions
train_identity.csv	26,529,680	Optional train identity
test_transaction.csv	613,194,934	Unlabeled transactions
test_identity.csv	25,797,161	Optional test identity

4.2 Join integrity

Row authority is preserved via a LEFT JOIN on `TransactionID` (Table 4.2).

Table 4.2: *Transaction–identity join audit*

Dataset	Transactions	Identity matches	Unmatched	Row difference
Train	590,540	144,233	446,307	0
Test	506,691	141,907	364,784	0

4.3 Leakage-controlled transformation order

Transformations fit strictly on training partitions and apply forward to validation and holdout. Figure 4.1 details the execution pipeline.



Figure 4.1: Detailed data processing pipeline. Generates six canonical dataset partitions: `train_original`, `train_weighted`, `train_balanced`, `validation`, `holdout`, and `kaggle_test`.

4.4 Chronological split

Sorting by `TransactionDT` creates non-overlapping chronological splits (Table 4.3).

Table 4.3: Chronological labeled splits

Dataset	Rows	Fraud	Fraud rate	TransactionDT range
Train	412,956	14,522	3.5166%	86,400–10,432,902
Validation	88,486	3,036	3.4310%	10,432,915–13,136,583
Holdout	89,098	3,105	3.4849%	13,136,627–15,811,131

4.5 Feature engineering & class balance

The canonical **68**-feature vector includes Time, Amount, Missingness, Anonymized behavior (C1--C14, D1--D15), Historical entity aggregates, and Categoricals.

Table 4.4: Training and evaluation class distributions

Dataset	Legitimate	Fraud	Ratio
<code>train_original</code>	398,434	14,522	27.44
<code>train_weighted</code>	398,434	14,522	27.44
<code>train_balanced</code>	43,872	14,522	3.02
<code>validation</code>	85,450	3,036	28.15
<code>holdout</code>	85,993	3,105	27.70

4.6 Verification result

The automated verifier passes **94** checks with zero failures, confirming handoff readiness:

```
ready_for_downstream_training
```

CHAPTER 5

MODEL DEVELOPMENT, EVALUATION, AND SERVING

5.1 Artifact identity and terminology

The project evaluates four model states: Historical V2 (LightGBM), Serving Default V2 (Configured champion), and Current Candidate (CatBoost Balanced, status: `not_promoted`) (Table 5.1).

Table 5.1: *Model identifiers and runtime roles*

Identifier	Model	Role	Status
Historical V2	LightGBM	Historical artifact	Historical
Serving default _{v2}	LightGBM	Configured champion	Unchanged
Current candidate	CatBoost balanced	Challenger	Not promoted

5.2 Model-development protocol

Architecture operates via **distributed Spark data preparation with single-node Python estimator training**. Figure 5.1 illustrates the end-to-end training lifecycle.

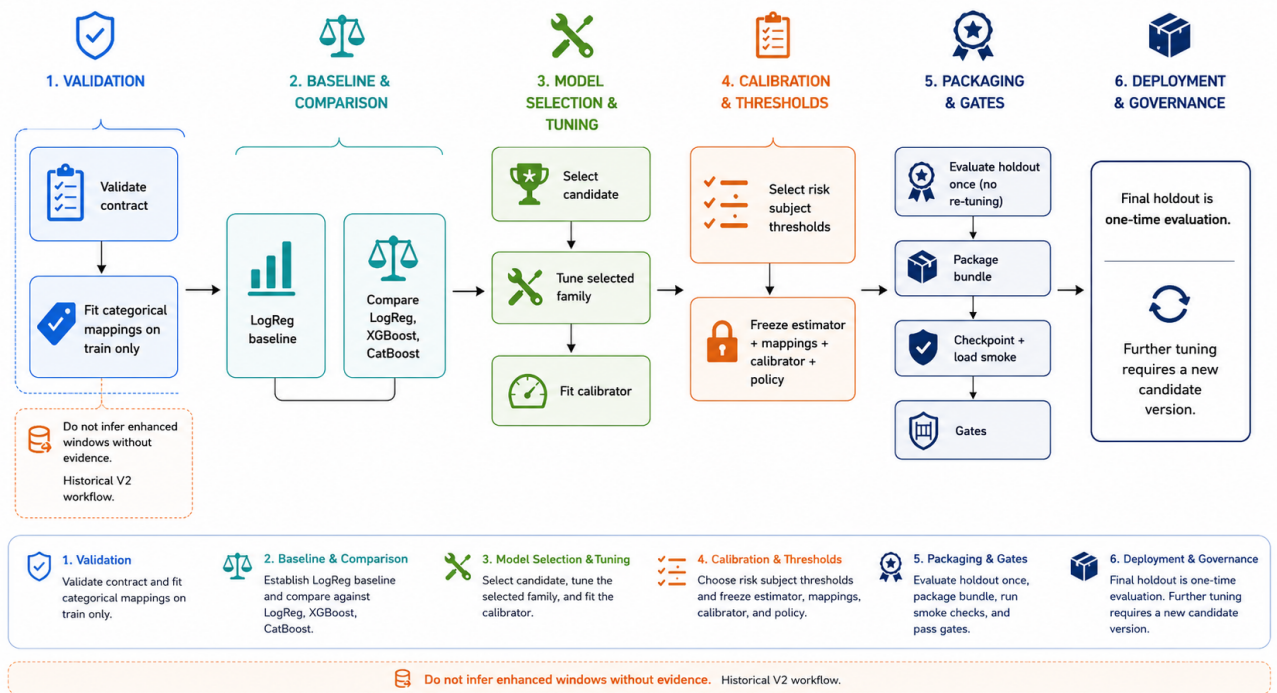


Figure 5.1: *End-to-end model training lifecycle. Illustrates model selection, tuning, fitting the calibrator within the Calibration & Thresholds stage, and holdout evaluation.*

5.3 PySpark MLlib DecisionTree baseline

To satisfy BDA501 big-data ML baseline requirements, a PySpark MLlib `DecisionTreeClassifier` pipeline evaluates the 68-feature vector across three dataset variants (Table 5.2).

Table 5.2: *PySpark MLlib DecisionTree Baseline Performance*

Model variant	Split	PR-AUC	ROC-AUC	Precision	Recall	Time (s)
baseline_tree	Validation	0.0205	0.2329	0.6928	0.1574	13.5
baseline_tree	Holdout	0.0209	0.2366	0.6641	0.1948	13.5
weighted_tree	Validation	0.2666	0.6866	0.1564	0.6400	12.1
weighted_tree	Holdout	0.2773	0.6984	0.1494	0.6583	12.1
undersampled_tree	Validation	0.0284	0.4265	0.2791	0.5217	5.7
undersampled_tree	Holdout	0.0278	0.4163	0.2039	0.5076	5.7

5.4 Candidate models and selection

Evaluation compares Logistic Regression, LightGBM, XGBoost, and CatBoost (Table 5.3). CatBoost Balanced achieved highest PR-AUC.

Table 5.3: *Current model-family selection evidence*

Model	Training variant	ROC-AUC	PR-AUC	Decision
Logistic Regression	Weighted	0.8219	0.3090	Baseline
LightGBM	Weighted	0.9116	0.5682	Compared
XGBoost	Weighted	0.8879	0.5529	Compared
CatBoost	Weighted	0.8757	0.5245	Compared
CatBoost	Balanced	0.9070	0.5703	Selected

5.5 Current CatBoost candidate results

Table 5.4 records candidate performance across validation and holdout splits.

Table 5.4: *Current CatBoost candidate validation and holdout metrics*

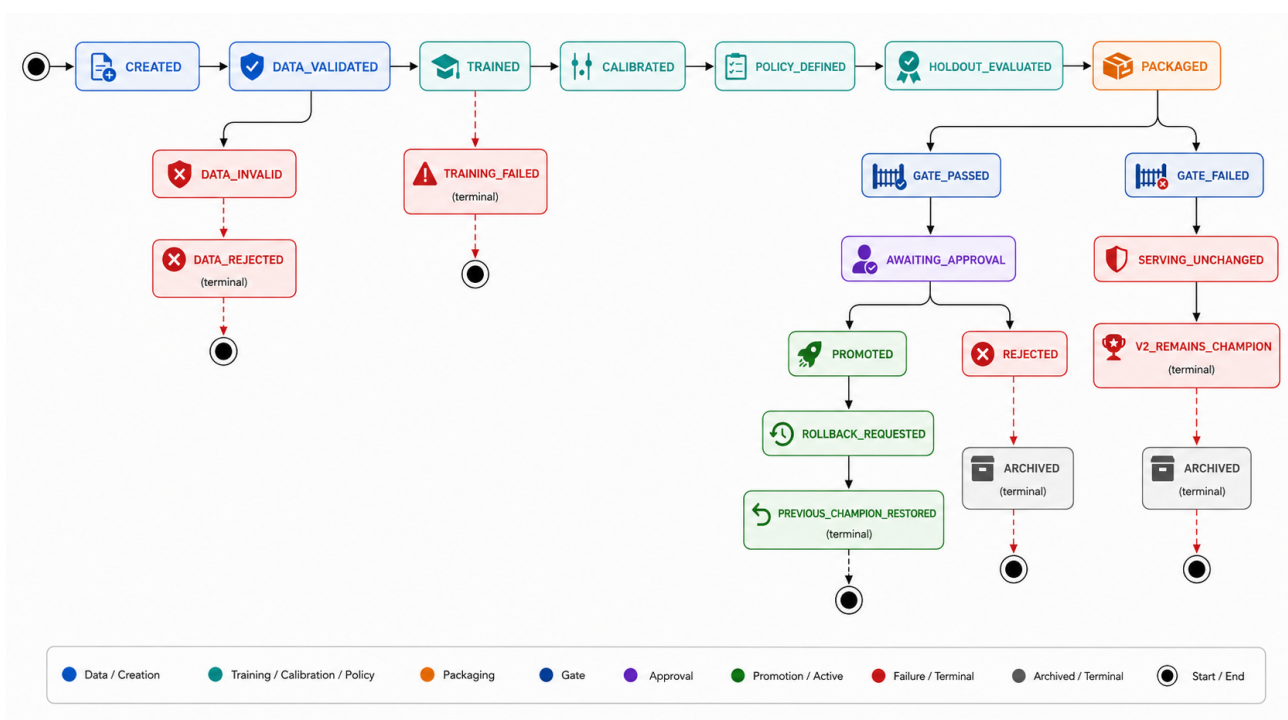
Window	Threshold	PR-AUC	ROC-AUC	Precision	Recall	Brier
Validation selection	0.50	0.5806	0.9081	0.3970	0.6220	–
Holdout	0.16	0.4266	0.8781	0.3820	0.5182	0.0248

5.6 Packaging, checksum, and promotion

The candidate promotion gate evaluated 9 criteria (Table 5.5). Due to holdout PR-AUC degradation (0.4266 vs target 0.4582), the promotion decision remains `not_promoted`.

Table 5.5: *Current candidate promotion gate*

Gate	Result	Evidence
Holdout PR-AUC ≥ 0.4582	Fail	0.4266
Holdout ROC-AUC ≥ 0.8746	Pass	0.8781
Review precision ≥ 0.30	Pass	0.3820
Calibration non-regression	Pass	Brier decreased
Artifact checksum	Pass	SHA-256 match
Artifact-load smoke	Pass	Manifest verified
Data contract verification	Pass	94 checks ready
Version match	Pass	v2
Git clean	Fail	git_dirty=true

**Figure 5.2:** *Promotion state machine. A failed candidate enters a failure/archive path while serving configuration remains unchanged.*

5.7 Request-time scoring

Real-time scoring maps categoricals via persisted indexers, applies isotonic calibration, and computes TreeSHAP feature attributions [5]. Responses support `full_feature` and `partial_demo` modes.

CHAPTER 6

APPLICATION AND DEPLOYMENT

6.1 Integrated serving architecture

The application operates as an **in-process model-serving modular monolith**. FastAPI imports the scoring package directly, persisting scored transactions in PostgreSQL/SQLite and serving a React dashboard.

6.2 Scoring and review workflow

The backend returns probability, risk score (**0–100**), risk band, decision, and TreeSHAP attributions. Scored transactions and human analyst reviews are stored as distinct database entities.

6.3 Frontend interface

Figures 6.1–6.2 present the real operational dashboard and analyst review workflow.

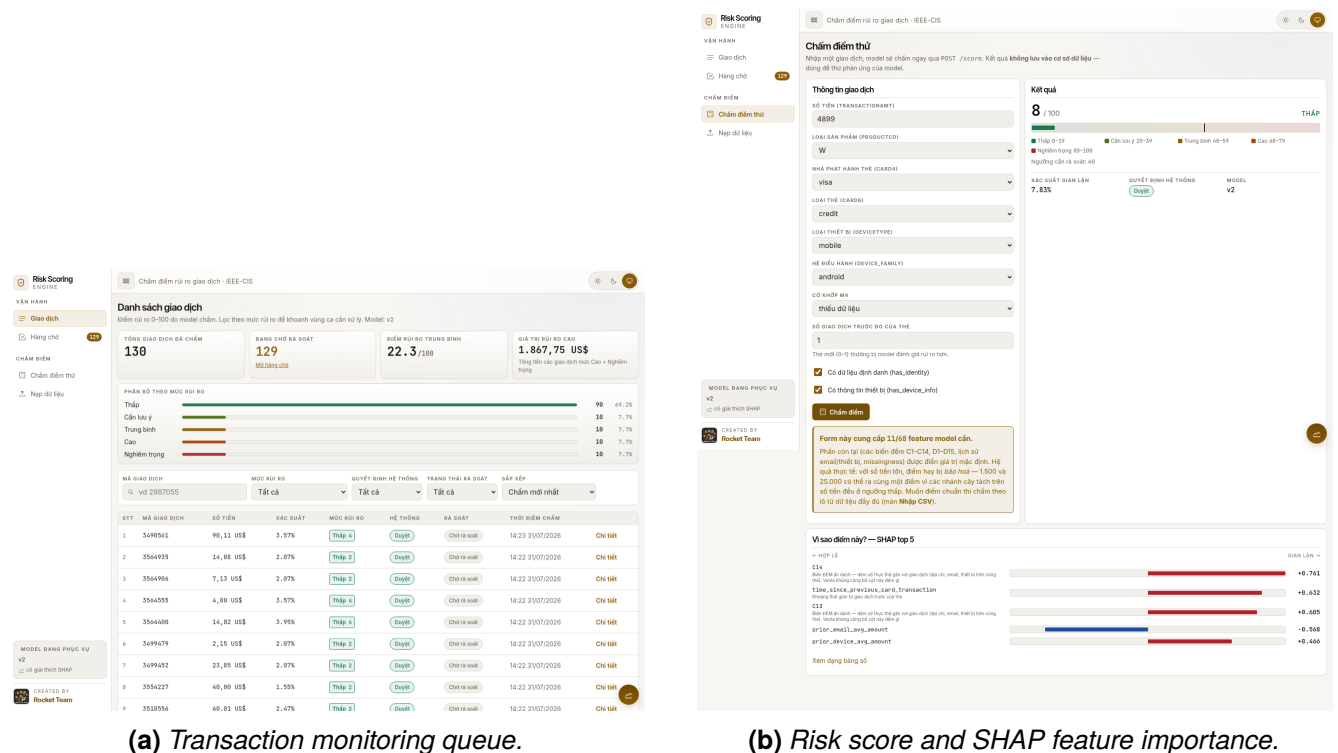
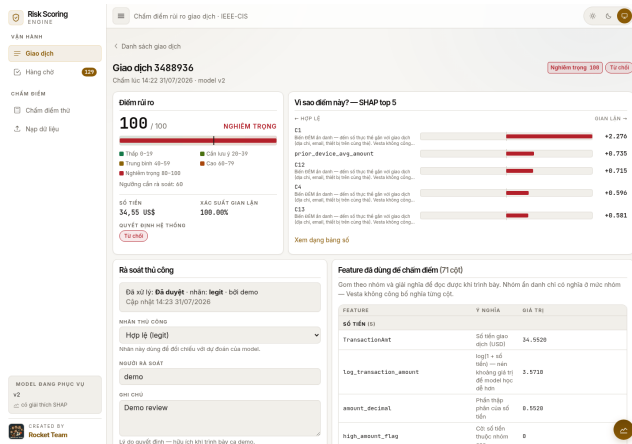
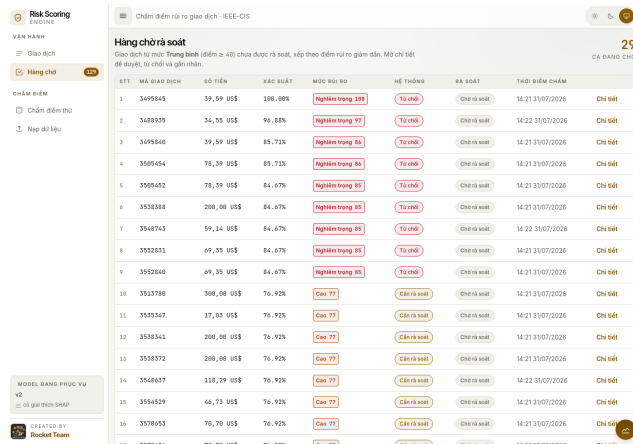


Figure 6.1: Application User Interface: Transaction queue and real-time risk scoring breakdown.



(a) Transaction detail panel.



(b) Analyst fraud review workflow.

Figure 6.2: Application User Interface: Transaction inspection and analyst review interface.

6.4 Deployment boundary

Docker Compose orchestrates preprocessing, training, backend, and React UI profiles, mounting processed Parquet artifacts into the backend container.

CHAPTER 7

VALIDATION AND WHOLE-SYSTEM REVIEW

7.1 Evidence status

The verifier passes **94** validation checks with status `ready_for_downstream_training`. Saved artifacts provide empirical evidence for data contracts, baseline comparisons, and candidate holdout evaluation.

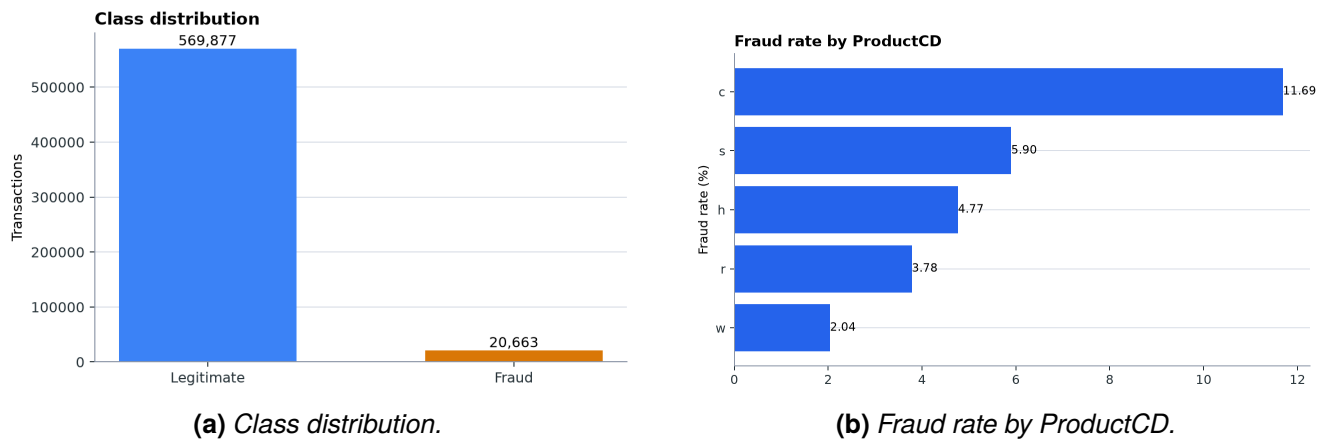
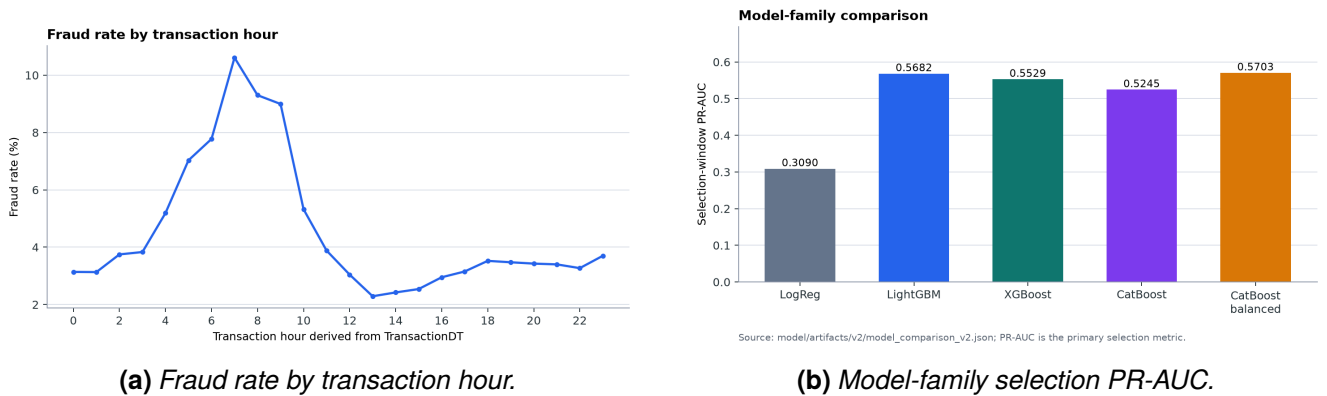


Figure 7.1: Exploratory Data Analysis: Class distribution and fraud rate by ProductCD.



(a) Fraud rate by transaction hour.

(b) Model-family selection PR-AUC.

Figure 7.2: Temporal fraud association and model-family performance selection.

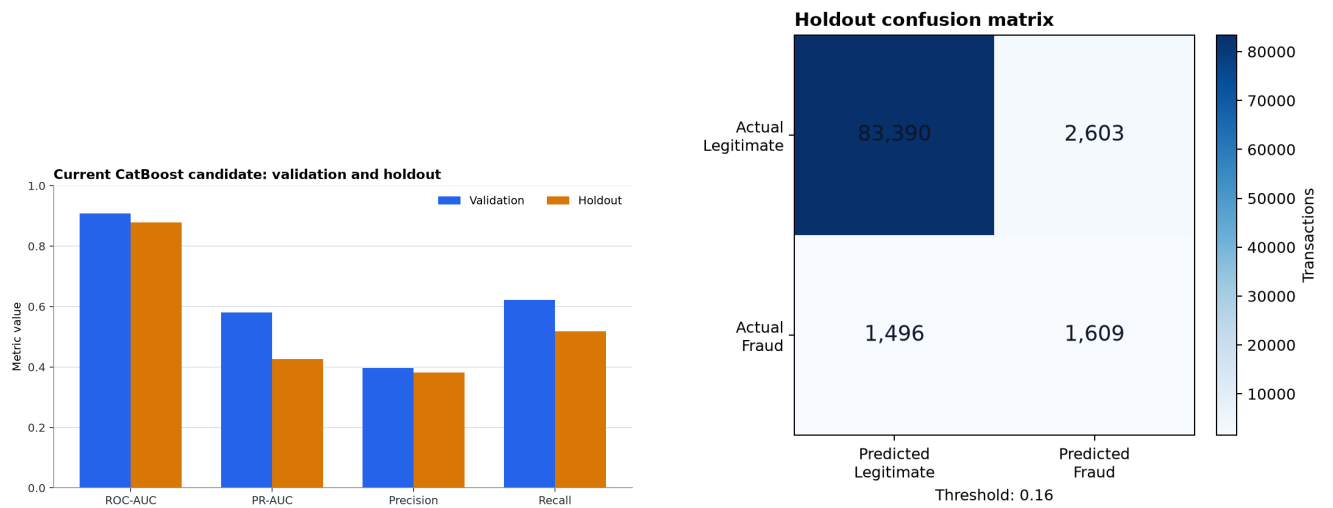


Figure 7.3: Candidate holdout evaluation: Metrics drop and operating confusion matrix.

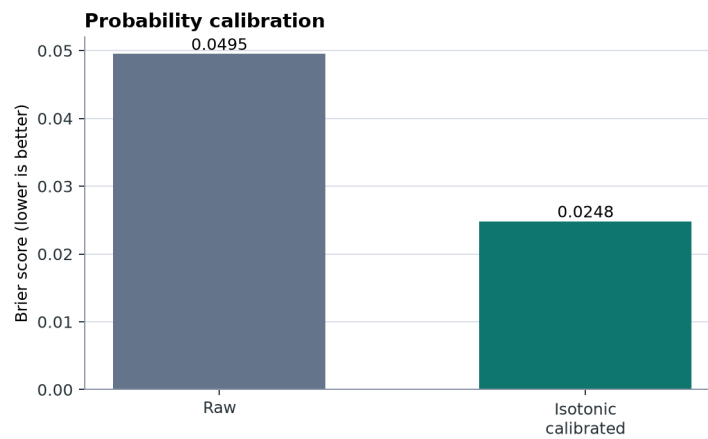


Figure 7.4: Probability calibration: Raw vs isotonic-calibrated Brier score on holdout.

7.2 Technical challenges and implemented solutions

Table 7.1 details the technical challenges, solutions, and repository evidence.

Table 7.1: *Technical Challenges, Implemented Solutions, and Evidence*

Challenge	Implemented solution	Evidence
Transaction–identity sparsity	Transaction-authoritative LEFT JOIN pre-serving 100% transaction rows with missingness indicators	Preprocessing Join Audit Step
Missing values	Train-only median imputation for numeric; explicit 'missing' category	Feature Contract Specification
Class imbalance	Evaluated original, weighted (weight 3.0), and balanced undersampled splits	Balanced Parquet Artifact
Temporal leakage	Chronological split on TransactionDT (train → val → calib → holdout)	Leakage-free Pipeline Stages
Schema & feature order	Frozen canonical 68-feature vector and machine-checked schema hash gate	Verification Engine Gate
Verifier failure	Automated validation gate outputting ready_for_downstream_training	Verification Report Artifact
Spark-to-pandas boundary	Spark handles distributed data preparation; Python fits single-node matrices	Model Execution Logs
Offline–online feature mismatch	Standardized backend feature loader mapping scoring request vector to schema	Backend Feature Loader

7.3 Consolidated limitations

1. **Reproducibility:** Local MLflow tracking and single-node pandas training.
2. **Policy Alignment:** Runtime backend risk bands decoupled from offline threshold optimization.
3. **Offline-Online Parity:** Real-time feature store for historical aggregates not implemented.
4. **Operational Controls:** Fail-open fallback heuristic and in-memory background jobs.
5. **Security & Governance:** Authentication, role-based authorization, and durable audit logs absent.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The project demonstrates an end-to-end fraud scoring system separating distributed Spark data preparation from single-node Python model training. CatBoost Balanced achieved validation ROC-AUC **0.9081** and PR-AUC **0.5806** (holdout ROC-AUC **0.8781**, PR-AUC **0.4266**), while candidate status remains `not_promoted`.

8.2 Summary of research question answers

- **RQ1 (Data Integrity):** Leakage-free chronological splits and schema contracts are verified.
- **RQ2 (Classifier Performance):** CatBoost achieved highest PR-AUC; performance degrades gracefully on holdout.
- **RQ3 (Serving Architecture):** In-process Python model package enables Spark-free real-time scoring.
- **RQ4 (System Boundaries):** Architectural limits (offline-online parity, fail-open fallback) bound the demo.

8.3 Future work

1. Implement an online feature store for real-time aggregation parity.
2. Separate runtime policy configuration from offline model weights.
3. Integrate production audit logging, security, and automated drift monitoring.

BIBLIOGRAPHY

- [1] IEEE Computational Intelligence Society, “leee-cis fraud detection,” 2019.
- [2] Amazon Science, “Fraud dataset benchmark: leee-cis fraud detection,” 2023.
- [3] J. Davis and M. Goadrich, “The relationship between precision-recall and roc curves,” in *Proceedings of the 23rd International Conference on Machine Learning*, pp. 233–240, 2006.
- [4] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.
- [5] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

CHAPTER A

API REFERENCE

A.1 Common enums

```
RiskBand = low | guarded | medium | high | critical
Decision = approve | review | reject
ReviewState = pending | approved | rejected
ReviewLabel = fraud | legit
ScoringMode = full_feature | partial_demo
```

A.2 Core response fields

Table A.1: Core API fields

Field	Type	Meaning
fraud_probability	float	Model/fallback output in $[0, 1]$
risk_score	integer	Rounded probability times 100
risk_band	enum	Fixed backend score band
decision	enum	Automatic backend policy result
scoring_mode	enum	Full vector or partial demonstration
shap_top5	list/null	Local model contributions when available
model_version	string	Artifact version used for the score
processing_version	string/null	Data processing lineage
feature_schema_version	string/null	Feature contract lineage

A.3 Endpoint summary

Method	Path	Response
GET	/health	Process status
GET	/meta	Model and feature metadata
POST	/score	Score response
GET	/transactions	Paginated transactions
GET	/transactions/stats	Aggregate statistics
GET	/transactions/import/template	UTF-8 CSV
POST	/transactions/import	Import outcome and coverage
GET	/transactions/{id}	Transaction detail
POST	/transactions/{id}/review	Updated detail
GET	/datasets	Available datasets

Method	Path	Response
POST	/data/load	Background job
GET	/jobs/{id}	Job state
GET	/jobs/active/current	Job or null
POST	/jobs/{id}/cancel	Updated job

The normative, implementation-aligned contract is maintained in the repository API contract specification.

CHAPTER B

DEPLOYMENT AND IMPLEMENTATION DIAGRAMS

B.1 Application deployment

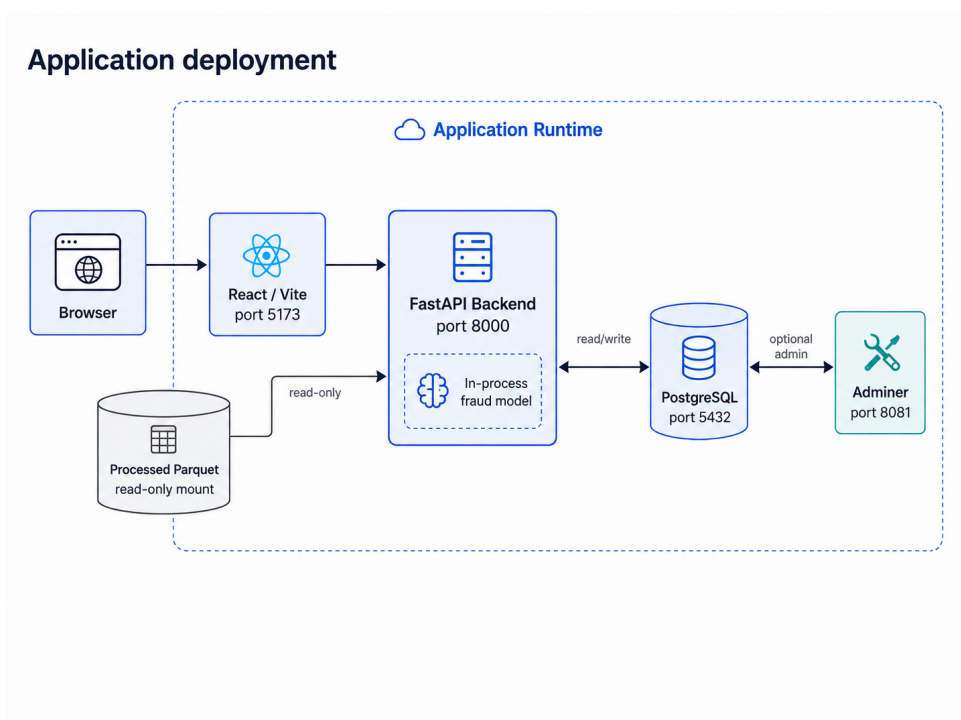


Figure B.1: Default application deployment. Browser, React/Vite, FastAPI backend, PostgreSQL, and in-process fraud model.

The application diagram is implementation-level evidence for the modular monolith. Spark pre-processing and Python model training are separate profiles; the diagram does not imply distributed estimator fitting.

CHAPTER C

FEATURE AND RISK CONTRACTS

C.1 Canonical feature groups

Table C.1: *Canonical feature groups*

Group	Representative features
Amount	TransactionAmt, log/capped/decimal amount, amount band, card-mean ratios and deviations
Time	TransactionDT, day, week, hour, day-of-week proxy, age, night flag
Missingness	Selected/identity missing counts and ratios
Presence	Identity, device, payer/recipient email, distance, address, same-domain flags
Anonymous behavior	dist1, dist2, selected C1--C14, selected D1--D15
Card history	Prior count/sum/mean/stddev and time since previous transaction
Email history	Prior count/sum/mean
Device history	Prior count/sum/mean
Categorical	ProductCD, card4, card6, DeviceType, device_family, M4, amount_band

C.2 Forbidden model columns

```
TransactionID
isFraud
class_weight
split_name
processing_version
feature_schema_version
generated_at
```

C.3 Risk bands

Minimum	Maximum	Band	Decision
0	19	low	approve
20	39	guarded	approve
40	59	medium	review
60	79	high	review
80	100	critical	reject

The risk bands are a fixed demonstration policy. They are not derived from the saved V2 validation operating point.

CHAPTER D

REPRODUCIBLE RUNBOOK

D.1 Raw data placement

```
data/data/ieee-fraud-detection/  
train_transaction.csv  
train_identity.csv  
test_transaction.csv  
test_identity.csv  
sample_submission.csv # optional
```

D.2 Preprocessing

```
docker compose -f docker-compose.preprocessing.yml build  
docker compose -f docker-compose.preprocessing.yml run --rm preprocess  
docker compose -f docker-compose.preprocessing.yml run --rm verify-  
processed
```

If a completed Spark export lacks a finalized manifest:

```
python -m pipeline.processed_contract \  
--output-dir data/processed/ieee_cis_fraud_risk  
python -m pipeline.verify_processed_data \  
--output-dir data/processed/ieee_cis_fraud_risk
```

D.3 Model training

```
bash scripts/run_official_full.sh
```

The official workflow runs preprocessing, contract validation, baseline, comparison, tuning, calibration, threshold analysis, holdout evaluation, packaging, checksum generation, and the promotion gate. The current official decision is `not_promoted`; the serving configuration remains unchanged. Holdout evaluation must occur only after model and policy selection are fixed.

D.4 Application

```
docker compose up --build
```

Then open:

- **dashboard:** <http://localhost:5173>;
- **API documentation:** <http://localhost:8000/docs>;

- optional Adminer: `http://localhost:8081`; select the PostgreSQL system, then use the configured database credentials.

D.5 Local development

```
# Terminal 1
cd backend
uv sync --frozen
uv run uvicorn fraud_backend.main:app --reload

# Terminal 2
cd frontend
npm ci
npm run dev
```

D.6 Rollback

```
FRAUD_MODEL_SERVING_VERSION=v1
```

Restart the backend and verify `/meta` before accepting traffic. The rollback setting is a configuration path, not evidence that V1 has been validated as the current champion in the official snapshot.

CHAPTER E

TEST AND CONTRIBUTION MATRIX

E.1 Recommended validation commands

```
# Processed data (read-only unless --write-report is supplied)
python -m pipeline.verify_processed_data \
    --output-dir data/processed/ieee_cis_fraud_risk

# Model
cd model
uv run ruff check .
uv run pytest -q

# Backend
cd backend
uv run ruff check src tests
uv run pytest -q

# Frontend
cd frontend
npm run lint
npm run build
python scripts/check-ui.py
```

E.2 Acceptance scenarios

Table E.1: *Acceptance scenarios*

Scenario	Expected evidence
Processed handoff	Verifier passes every schema, row, split, and artifact check
Real model startup	<code>/meta</code> returns expected version with no warning
Full-feature score	<code>scoring_mode=full_feature</code> ; probability bounded
Partial score	<code>partial_demo</code> is visible; required fields enforced
Unseen category	Request succeeds with training-defined unknown mapping
SHAP absent	UI renders explicit unavailable state
Review update	Status/label persist and queue/statistics invalidate

Scenario	Expected evidence
CSV row error	Good rows commit; bad row appears in bounded error list
Job reconnect	Active job is returned after browser refresh
Model rollback	V1 loads and <code>/meta</code> reports V1

E.3 Team roster and repository role mapping

Table E.2 details the official team roster, roles, and repository responsibilities for RocketUpTeam.

Table E.2: *RocketUpTeam Roster and Repository Role Mapping*

Member Name	Role	Primary Responsibility
TUYEN Le Quang	Project Leader	Responsible for system ideation and overall project management.
NHI Nguyen Le Hong	Content & Presentation Coordinator	Responsible for synthesizing project content, designing slides, and developing the presentation strategy.
AN Duong Binh	Big Data & Pipeline Engineer	Responsible for Spark data ingestion, left joins, temporal splitting, and feature contract verification.
LONG Pham Duc	Machine Learning Engineer	Responsible for classifier training, hyperparameter tuning, isotonic calibration, and metric evaluation.
TRUNG Do Quoc	Backend & API Engineer	Responsible for FastAPI scoring service, database persistence, and scoring contracts.
QUAN Duong Hong	Frontend & UI Engineer	Responsible for React dashboard UI, risk visualizations, and analyst review workflow.